

CONTENTS

M3 — Ejercicios Adicionales de Práctica

Memorias ROM · Flip-Flops · Contadores · Memorias RAM — Arquitectura de Computadoras (FING UdelaR)

Índice de dificultad

1. ROM — Capacidad y notación

Ejercicio 1 (● fácil)

Ejercicio 2 (● difícil — combinación en ambos ejes con decodificador)

2. ROM como traductor de una función dada

Ejercicio 3 (● fácil)

Ejercicio 4 (● medio — traductor con salida multi-bit)

3. Flip-Flops y tablas de excitación

Ejercicio 5 (● medio — construcción de un tipo de FF a partir de otro)

Ejercicio 6 (● difícil — diseño de circuito secuencial completo con tabla de excitación)

4. Contadores

Ejercicio 7 (● medio — diseño de un contador módulo-N con FF JK)

Ejercicio 8 (● difícil — análisis de un contador dado, con anomalía a detectar)

Preguntas teóricas rápidas tipo parcial

Resumen ultra-rápido de este documento

M3 — Ejercicios Adicionales de Práctica

MEMORIAS ROM · FLIP-FLOPS · CONTADORES · MEMORIAS RAM — ARQUITECTURA DE COMPUTADORAS (FING UDELAR)

Este documento complementa la **Guía temática M3_Memorias_ROM.md**, la **Guía temática M4_Circuitos_Secuenciales.md** y el **Práctico 4 ya resuelto** (Práctico 4 (Terminado)/Practico4_Resolucion.md). Ningún ejercicio de acá repite números, funciones o tamaños ya usados en ese material — son **8 ejercicios nuevos**, con la misma metodología y el mismo nivel de rigor (verificación explícita en cada uno, con un script de Python que corre las 8 verificaciones de punta a punta).

Índice de dificultad

#	Ejercicio	Tema	Dificultad
1	ROM $2^{17} \times 24$ — capacidad y notación	ROM: capacidad	 Fácil
2	De chips $4K \times 8$ a una ROM de $64K \times 32$	ROM: combinación con decodificador (ambos ejes)	 Difícil
3	ROM traductora: función mayoría de 3 variables	ROM: traductor de función lógica	 Fácil
4	ROM traductora: BCD $\rightarrow$ código Aiken (2421)	ROM: traductor de código, salida multi-bit	 Medio
5	Construir un FF S-R síncronico a partir de un FF T	Flip-flops: construcción por ecuación característica	 Medio
6	Detector de “11” con solapamiento usando un único FF JK	Flip-flops: tabla de excitación, diseño de circuito secuencial	 Difícil
7	Contador módulo 5 (0-4) con FF JK	Contadores: diseño con tabla de excitación y don't cares	 Medio
8	Analizar un contador de 3 bits con una ecuación “sospechosa”	Contadores: análisis de un circuito dado	 Difícil

Todas las verificaciones numéricas, tablas de verdad y simulaciones de este documento fueron corridas con un único script de Python (`verify_m3.py`) que evalúa las 8 fórmulas/tablas de forma exhaustiva contra su propia especificación — el resultado íntegro de esa corrida se resume en el bloque “ Verificación” de cada ejercicio.

1. ROM — Capacidad y notación

EJERCICIO 1 (● FÁCIL)

Un chip de ROM se especifica como $2^{17} \times 24$. Calculá: (a) cantidad de palabras N, (b) líneas de dirección, (c) líneas de datos, (d) capacidad total en bits, y (e) capacidad total en KB.

Resolución:

- **(a)** $N = 2^{17} = 131072$ palabras (ya viene como potencia de 2, no hace falta redondear).
- **(b)** Líneas de dirección = $\log_2(N) = \log_2(2^{17}) = 17$ líneas ($A_0..A_{16}$), por identidad directa (regla de la Sección 2 de la guía temática M3).
- **(c)** Líneas de datos = $M = 24$ líneas ($D_0..D_{23}$) — no depende de N.
- **(d)** Capacidad total = $N \times M = 131072 \times 24 = 3145728$ bits.
- **(e)** En KB: $3145728 \text{ bits} / 8 = 393216 \text{ bytes} = 393216/1024 = 384$ KB.

✓ **Verificación** (script, bloque EJ1): `N=131072 M=24 cap_bits=3145728 cap_bytes=393216 = 384.0 KB`. Coincide exactamente con el cálculo manual; además $393216/1024 = 384$ sin resto, confirmando que la capacidad en KB es un número entero (esperable porque 24 no es potencia de 2 pero el producto final sí resulta múltiplo exacto de 1024×8).

EJERCICIO 2 (● DIFÍCIL — COMBINACIÓN EN AMBOS EJES CON DECODIFICADOR)

Se dispone de chips de ROM de $4K \times 8$ bits (con CS y OE estándar). Se pide construir una ROM de $64K \times 32$ bits. Diseñá el esquema completo: cantidad de chips, decodificador necesario, qué líneas van en paralelo y cuáles al decodificador, y cómo se arman las salidas de datos.

Resolución — identificar qué eje cambia: N sube de 4K a 64K, y M sube de 8 a 32 → cambian **ambos ejes** → caso general (Sección 3.c de la guía temática): grilla de bancos (eje N) $\times$ paralelo (eje M).

Paso 1 — eje N (cantidad de palabras). Bancos necesarios = $64K / 4K = 65536/4096 = 16$ bancos. Líneas de dirección internas de cada chip = $\log_2(4096) = 12$ ($A_0..A_{11}$). Líneas de dirección totales de la ROM final = $\log_2(65536) = 16$. Líneas extra para el decodificador = $16 - 12 = 4$ líneas ($A_{12}..A_{15}$) → decodificador $4 \rightarrow 16$, cuyas 16 salidas activan el CS de cada uno de los 16 bancos (una sola a la vez).

Paso 2 — eje M (ancho de palabra). ROMs en paralelo por banco = $32/8 = 4$ chips (mismas 12 líneas de dirección internas y mismo CS dentro del banco; salidas concatenadas: $D_0-D_7, D_8-D_{15}, D_{16}-D_{23}, D_{24}-D_{31}$).

Paso 3 — total de chips = 16 bancos × 4 en paralelo = **64 ROMs de 4K×8**.

Circuito: las 12 líneas menos significativas ($A_0..A_{11}$) van en paralelo a los 64 chips; las 4 líneas más significativas ($A_{12}..A_{15}$) van al decodificador 4→16, que habilita el CS de los 4 chips de un banco a la vez (los otros 60 quedan en alta impedancia); dentro del banco activo, cada uno de los 4 chips aporta 8 bits distintos del bus de 32 bits de salida.

✓ **Verificación** (script, bloque EJ2): `banks=16 par=4 total_chips=64`. Chequeo de capacidad: `64 chips × (4096×8) = 2097152 bits`, y `65536×32 = 2097152 bits` — **coinciden exactamente** (`cap check: True`), confirmando que no sobra ni falta capacidad. Chequeo de líneas: `addr_final=16 = internal(12) + decoder(4)` ✓.

Error frecuente de tolerancia cero (igual que en la guía M3, Sección 3): unir las 64 salidas con compuertas OR, o repartir mal cuáles líneas van al decodificador y cuáles en paralelo. Regla fija: **las líneas de MÁS peso van siempre al decodificador (seleccionan el banco)**, las de MENOS peso van en paralelo a todos los chips de un mismo banco (son la dirección “interna”).

2. ROM como traductor de una función dada

EJERCICIO 3 (● FÁCIL)

Se quiere implementar con una ROM la función **mayoría de 3 variables**: $F(A,B,C) = 1$ si y solo si al menos 2 de las 3 entradas valen 1 (es la función que usa, por ejemplo, un sistema de votación por triplicado o un corrector de errores simple). Determiná el tamaño mínimo de la ROM y volcá su contenido completo.

Resolución:

- Entradas: 3 (A, B, C) → $2^3 = 8$ direcciones.
- Salida: 1 bit (F) → **1 línea de datos**.
- **Tamaño mínimo: ROM de 8×1.**

Contenido completo (dirección $A_2A_1A_0 = A B C$):

Dirección (A B C)	Decimal	Cantidad de “1”	F
000	0	0	0
001	1	1	0
010	2	1	0
011	3	2	1
100	4	1	0
101	5	2	1
110	6	2	1
111	7	3	1

✓ **Verificación** (script, bloque EJ3): se generaron las 8 filas evaluando $F=1$ si $(A+B+C) \geq 2$ y coinciden exactamente con la tabla de arriba, fila por fila (por ejemplo, dirección 011: A=0,B=1,C=1 → dos unos → F=1 ✓; dirección 100: A=1,B=0,C=0 → un solo uno → F=0 ✓).

EJERCICIO 4 (🟡 MEDIO — TRADUCTOR CON SALIDA MULTI-BIT)

Se necesita una ROM que traduzca un dígito **BCD** (4 bits, 0 a 9) a su equivalente en **código Aiken (2421)**, un código ponderado usado en algunos sistemas de conteo BCD donde los pesos de los 4 bits son 2,4,2,1 en vez de 8,4,2,1. Determiná tamaño, organización, y volcá el contenido completo de la ROM (marcando qué direcciones son “no importa”).

Resolución:

- Entradas: 4 bits (dígito BCD) → $2^4 = 16$ direcciones (10 usadas, 0-9; 6 no usadas/no importa, 10-15, porque BCD nunca genera esos patrones).
- Salidas: 4 bits (código Aiken) → **4 líneas de datos**.
- **Tamaño: ROM de 16×4** como mínimo.

Contenido completo:

Dirección BCD	Dígito	Código Aiken (2421)
0000	0	0000
0001	1	0001
0010	2	0010
0011	3	0011
0100	4	0100
0101	5	1011
0110	6	1100
0111	7	1101
1000	8	1110
1001	9	1111
1010–1111	(no usado)	XXXX

Dato de diseño que explica los “saltos” en la tabla a partir del 5: el código Aiken 2421 se construye para que los dígitos 5 a 9 sean el **complemento a 1 bit a bit** del código del dígito (9–dígito). Por eso 5 (complemento de 9–5=4) no es simplemente “0101” como en BCD natural, sino el complemento de 0100, que es 1011. Esta propiedad de autocomplementación es la razón práctica por la que el código Aiken se usa en algunos contadores BCD que necesitan generar el complemento a 9 fácilmente.

✅ **Verificación** (script, bloque EJ4): se generó la tabla completa 0-9 y se verificó cruzadamente que, para cada dígito d de 5 a 9, el código guardado es efectivamente el complemento bit a bit del código de (9– d): por ejemplo $d=7 \rightarrow$ código 1101, y el complemento de $9-7=2 \rightarrow$ código(2)= 0010 $\rightarrow$ complemento= 1101 $\rightarrow$ **coincide=True**. Se verificó así para los 5 dígitos (5,6,7,8,9), con resultado True en los 5 casos.

3. Flip-Flops y tablas de excitación

EJERCICIO 5 (🟡 MEDIO — CONSTRUCCIÓN DE UN TIPO DE FF A PARTIR DE OTRO)

Se dispone únicamente de Flip-Flops tipo **T**. Se pide construir, a partir de uno de ellos, un Flip-Flop **S-R sincrónico** (con la combinación $S=R=1$ prohibida). Encontrá la expresión combinacional que

hay que anteponer a la entrada T.

Resolución — método de igualar ecuaciones características (Sección 5.1 de la guía M4):

1. Ecuación característica del FF deseado (S-R): $Q_{n+1} = S + R' \cdot Q_n$ (si $S=1 \rightarrow$ set; si $S=0, R=0 \rightarrow$ mantiene Q; si $S=0, R=1 \rightarrow$ reset; $S=R=1$ prohibido).
2. Ecuación característica del FF disponible (T): $Q_{n+1} = Q_n \oplus T$.
3. Igualando: $Q_n \oplus T = S + R' \cdot Q_n \rightarrow$ despejo T construyendo la tabla de $T = Q_n \oplus Q_{n+1}$ para cada combinación permitida de (S,R, Q_n):

S	R	Q_n	Q_{n+1} (según S-R)	$T = Q_n \oplus Q_{n+1}$
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	0	1
1	0	0	1	1
1	0	1	1	0
1	1	X	prohibido	X (don't care)

Agrupando los “1” de T (filas $S=0, R=1, Q=1$ y $S=1, R=0, Q=0$) y aprovechando los don't-care de $S=R=1$ para simplificar (fusionando con las filas $S=1, R=1, Q=0$ y $S=0, R=1, Q=1$ vecinas del mapa de Karnaugh), se llega a la expresión mínima:

$$T = S \cdot Q_n' + R \cdot Q_n$$

Conclusión: basta con armar el circuito combinacional $T = S \cdot Q' + R \cdot Q$ (dos AND + un OR + un NOT, usando la propia salida Q realimentada) y conectarlo a la entrada T de un FF T físico.

✅ **Verificación** (script, bloque EJ5, fuerza bruta sobre las 6 combinaciones permitidas de (S,R,Q), excluyendo $S=R=1$ por prohibida): para cada fila se calculó $T=S \cdot Q' + R \cdot Q$, se aplicó la ecuación característica del FF T ($Q_{n+1}=Q_n \oplus T$), y se comparó contra el Q_{n+1} esperado según la definición de S-R. Resultado: **las 6 filas coinciden exactamente** (OK=True en las 6, TODAS LAS FILAS COINCIDEN: True). Chequeo manual de una fila: $S=1, R=0, Q=0 \rightarrow T=1 \cdot 1 + 0 \cdot 0 = 1 \rightarrow$ FF T da $Q_{n+1}=0 \oplus 1 = 1$, que es lo esperado para “set” ✓.

EJERCICIO 6 (● DIFÍCIL — DISEÑO DE CIRCUITO SECUENCIAL COMPLETO CON TABLA DE EXCITACIÓN)

Diseña, usando un único Flip-Flop **JK**, un circuito Mealy con una entrada serial x que detecte la aparición de **dos “1” consecutivos** (patrón “11”), con salida $Y=1$ exactamente en el ciclo en que se completa el patrón, permitiendo solapamiento (por ejemplo, con la entrada “111” debe detectar dos veces: en la posición 2 y en la posición 3). Usá la tabla de excitación del JK para llegar a J y K.

Resolución:

Paso 1 — estados. Alcanza con **un solo bit de estado** Q : $Q=0$ significa “el último bit visto no fue 1 (o es el arranque)”; $Q=1$ significa “el último bit visto fue 1”.

Paso 2 — tabla de transiciones y salida (análisis exhaustivo de las 4 combinaciones (Q,x)):

Q	x	Q_{n+1}	Y
0	0	0	0
0	1	1	0
1	0	0	0
1	1	1	1 (se completó “11”; el propio bit que cierra el patrón puede ser el comienzo del siguiente, por eso permite solapamiento)

Notar que $Q_{n+1} = x$ siempre (el estado “recuerda” simplemente el último bit recibido), y la salida $Y = Q \cdot x$ (solo enciende cuando el bit anterior Y el actual son ambos 1).

Paso 3 — tabla de excitación del JK (Sección 5.2 de la guía M4) aplicada a cada transición $Q \rightarrow Q_{n+1}$:

Q	x	Q_{n+1}	J	K
0	0	0	0	X
0	1	1	1	X
1	0	0	X	1
1	1	1	X	0

Paso 4 — Karnaugh de J y K (variables Q, x), aprovechando los don't-care: en las dos filas con $Q=0$, J vale 0 y 1 según x (y K es X) $\rightarrow J = x$. En las dos filas con $Q=1$, K vale 1 y 0 según x (y J es X) $\rightarrow K = x'$.

$J = x \quad K = x' \quad Y = Q \cdot x$ (lógica combinacional de salida, no entra al FF)

✓ **Verificación** (script, bloque EJ6): se probaron las 4 combinaciones de (Q,x) , calculando $J=x$, $K=x'$, aplicando la ecuación característica del JK ($Q_{n+1} = J \cdot Q' + K' \cdot Q$) y comparando contra la especificación — **coinciden las 4 filas** (TODO COINCIDE: True). Además se simuló el circuito completo sobre la tira $x = 1,1,1,0,1,1,0,1,1,1$, obteniendo $Y = 0,1,1,0,0,1,0,0,1,1$: Y se enciende en las posiciones 2 y 3 (correspondientes al “111” inicial, detectado dos veces por solapamiento) y en las posiciones 6 y 10 (correspondientes a los dos “11” siguientes) — comportamiento coherente con la especificación de detección con solapamiento.

Comparación con el ejemplo de la guía M4 (detector de “1101”, 4 estados, 2 FF): acá alcanza con **1 solo FF** porque el patrón a detectar es mucho más corto (2 bits) y no hace falta distinguir “prefijos parciales” más largos — la cantidad de estados de un detector de secuencia crece con la longitud del patrón buscado, no es una cantidad fija.

4. Contadores

EJERCICIO 7 (● MEDIO — DISEÑO DE UN CONTADOR MÓDULO-N CON FF JK)

Diseñá un contador síncrono módulo 5, con secuencia **0,1,2,3,4,0,1,2,...**, usando Flip-Flops **JK**. Los estados 5, 6 y 7 (101,110,111) nunca deben ocurrir en operación normal — tratalos como no importa tanto en el próximo estado como en las entradas de los FF.

Resolución:

Paso 1 — bits necesarios: 5 estados $\rightarrow \lceil \log_2(5) \rceil = 3$ bits ($q_2q_1q_0$), con 3 combinaciones (5,6,7) sin usar.

Paso 2 — tabla de transiciones y excitación (JK, con don't-care en los 3 estados no usados):

Estado	$q_2q_1q_0$	próximo	$J_2 K_2$	$J_1 K_1$	$J_0 K_0$
0	000	001	0 X	0 X	1 X
1	001	010	0 X	1 X	X 1
2	010	011	0 X	X 0	1 X
3	011	100	1 X	X 1	X 1
4	100	000	X 1	0 X	0 X
5,6,7	101,110,111	—	X X	X X	X X

Paso 3 — Karnaugh de cada entrada (3 variables q_2, q_1, q_0 , aprovechando los don't-care de los estados 5-7 para simplificar al máximo):

$$J_0 = \overline{q_2} \cdot \overline{q_0} \quad K_0 = 1 \quad J_1 = \overline{q_2} \cdot q_0 \quad K_1 = q_0 \quad J_2 = q_1 \cdot q_0 \quad K_2 = 1$$

Chequeo manual de una fila (estado 3 = 011): $J_0 = (\neg 1)(\neg 1) = 0 \cdot 0 = 0$, $K_0 = 1 \rightarrow q_0: 1 \rightarrow (J=0, K=1) \rightarrow 0$. $J_1 = (\neg 1)(1) = 0$, $K_1 = 1 \rightarrow q_1: 1 \rightarrow (J=0, K=1) \rightarrow 0$. $J_2 = 1 \cdot 1 = 1$, $K_2 = 1 \rightarrow q_2: 0 \rightarrow (J=1, K=1, \text{toggle}) \rightarrow 1$. Próximo estado = 100 = 4 ✓ (coincide con la tabla: 011 → 100).

✓ **Verificación** (script, bloque EJ7): se simuló el contador completo, arrancando en 000 y aplicando las ecuaciones de J_i, K_i de arriba junto con la ecuación característica del JK durante 12 flancos de reloj.

Secuencia obtenida: [0, 1, 2, 3, 4, 0, 1, 2, 3, 4, 0, 1, 2] — **cicla exactamente**

0,1,2,3,4,0,1,2,3,4,... como pide el enunciado, sin nunca visitar los estados 5, 6 o 7.

EJERCICIO 8 (● DIFÍCIL — ANÁLISIS DE UN CONTADOR DADO, CON ANOMALÍA A DETECTAR)

Un contador síncrono de 3 bits ($q_2q_1q_0$), construido con Flip-Flops JK, tiene las siguientes ecuaciones de excitación: $J_0=1, K_0=1 \quad J_1=q_0, K_1=q_0 \quad J_2=q_1 \cdot q_0, K_2=1$ Partiendo del estado inicial 000, determiná la secuencia de estados que genera el contador y decidí si es un contador binario módulo 8 correcto. Si no lo es, identificá exactamente cuál es el error de diseño.

Resolución:

Paso 1 — comparar con un contador binario módulo 8 estándar (FF JK). La forma estándar de un contador binario ascendente con JK es $J_i=K_i = \text{AND de todos los bits menos significativos}$ (para que el bit i -ésimo invierta exactamente cuando “hay acarreo” desde los bits menos significativos, como recuerda la guía M4, Sección 6.3): $J_0=K_0=1$ (bit 0 siempre invierte) ✓ coincide. $J_1=K_1=q_0$ (bit 1 invierte

cuando $q_0=1$) ✓ coincide. Pero para el bit 2, un contador binario estándar necesitaría $J_2=K_2=q_1 \cdot q_0$ (invierte solo cuando ambos bits menos significativos están en 1) — acá, en cambio, se da $K_2=1$ fijo, no $K_2=q_1 \cdot q_0$. Esto es sospechoso: $K_2=1$ significa que, cada vez que el bit 2 esté en $Q_2=1$ y se le pida “invertir o resetear” según J_2 , el $K=1$ fijo va a forzar $Q_2 \rightarrow 0$ sin condición, incluso cuando no correspondería.

Paso 2 — recalcular la tabla de transiciones paso a paso desde 000:

Estado actual	$q_2q_1q_0$	J_0K_0	J_1K_1	J_2K_2	próximo estado
0	000	1,1	0,0	0,1	001
1	001	1,1	1,1	0,1	010
2	010	1,1	0,0	0,1	011
3	011	1,1	1,1	1,1	100
4	100	1,1	0,0	0,1	001 ← no vuelve a 000

En el estado 100 ($q_2=1, q_1=0, q_0=0$): q_0 invierte ($J_0=K_0=1 \rightarrow \text{toggle} \rightarrow 0 \rightarrow 1$). q_1 se mantiene ($J_1=K_1=q_0=0 \rightarrow \text{mantiene} \rightarrow \text{sigue } 0$). q_2 : $J_2=q_1q_0=0$, $K_2=1$ fijo $\rightarrow$ con $J=0, K=1$ el JK **resetea** (va a 0) sin importar el valor anterior — es decir, **el bit 2 se apaga aunque “debería” quedarse en 1** (en un contador binario correcto, con $q_1q_0=00$, el bit 2 debería mantenerse en 1 hasta que le toque el próximo acarreo real). Esto produce el próximo estado **001**, no **000**: el contador nunca vuelve a pasar por el estado 000 después del primer ciclo.

Conclusión — no es un contador módulo 8 correcto. El bug está en K_2 : debería ser $K_2 = q_1 \cdot q_0$ (igual que J_2), no $K_2=1$ fijo. Con K_2 fijo en 1, el bit más significativo se resetea en cualquier flanco donde le toque “no cambiar” con $Q_2=1$ y ($J_1=K_1=0$ en ese instante para q_1) — en la práctica, el circuito termina atrapado en un ciclo de longitud 4 que **excluye permanentemente el estado 000** después del arranque.

✅ **Verificación** (script, bloque EJ8, simulación de 12 flancos desde 000): secuencia obtenida [0, 1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4]. Se confirma exactamente lo detectado a mano: después de llegar por primera vez a 4 (=100), el contador cae en el ciclo **1→2→3→4→1→2→3→4→...** (longitud 4), y **el estado 0 nunca vuelve a aparecer** — el script buscó explícitamente la primera repetición del estado inicial (0,0,0) en los 12 flancos simulados y no la encontró (longitud de ciclo hasta volver a 000: None), confirmando computacionalmente el diagnóstico manual.

Moraleja de tolerancia cero para “analizar un contador dado”: nunca asumas que las ecuaciones dadas forman un contador binario “estándar” solo porque se parecen. Siempre recalculá la tabla de transiciones completa a mano (o simulá) arrancando del estado inicial declarado, y prestá espe-

cial atención a los bits de **mayor** peso — ahí es donde suelen esconderse los errores de “K fijo en vez de $K=\text{producto de los bits menos significativos}$ ”, porque en los bits bajos ese error es menos visible (a menudo $J_0=K_0=1$ sí es correcto por casualidad, como pasó acá).

Preguntas teóricas rápidas tipo parcial

#	Pregunta	Respuesta
1	Una ROM se especifica como $2^{14} \times 20$. ¿Cuántas líneas de dirección y cuántas de datos tiene?	14 líneas de dirección ($\log_2(2^{14})=14$), 20 líneas de datos (M no depende de N)
2	¿Por qué en el Ejercicio 2 las líneas más significativas de la dirección van al decodificador y no en paralelo a todos los chips?	Porque esas líneas son las que distinguen en qué banco (grupo de chips) cae la dirección pedida; si fueran en paralelo, todos los bancos “creerían” tener la dirección activa a la vez, violando que solo un CS puede estar activo en cada instante
3	En la tabla de excitación de un FF JK, ¿por qué hay entradas “X” (no importa) y en la de un FF D no las hay?	Porque el JK tiene, para cada transición $Q(t) \rightarrow Q(t+1)$, más de una combinación de J,K que la produce (por ejemplo, $Q=0 \rightarrow Q(t+1)=0$ se logra con $J=0$ y $K=0$ o $K=1$); el FF D en cambio tiene una única entrada $D=Q(t+1)$, sin ambigüedad posible
4	¿Cuál es la diferencia esencial entre RAM y ROM en términos de “combinacional vs. secuencial”?	La ROM es combinacional (misma dirección $\rightarrow$ siempre mismo dato, sin importar el pasado); la RAM es secuencial , porque lo que se lee en una dirección depende de la última escritura hecha ahí — tiene memoria de su propia historia
5	Un contador de 6 estados (0 a 5) diseñado con FF, ¿cuántos Flip-Flops mínimos necesita, y qué se hace con los códigos sobrantes?	$\lceil \log_2(6) \rceil = 3$ Flip-Flops (2 no alcanzan: $2^2=4 < 6$); sobran 2 códigos (110, 111) que se tratan como don't care tanto en el próximo estado como en las entradas de los FF, para simplificar al máximo el Karnaugh
6	En el Ejercicio 8, ¿qué principio general viola la ecuación $K_2=1$ en un contador binario?	Viola la regla de que, en un contador binario sincrónico con FF JK, cada bit i debe tener $J_i=K_i=\text{AND de todos los bits menos significativos}$ — si K_i no coincide con J_i , el bit se “apaga” (resetea) en situaciones donde debería mantenerse, rompiendo la cuenta binaria correcta y produciendo un ciclo de longitud menor a la esperada

Resumen ultra-rápido de este documento

- **ROM — capacidad:** N =palabras, M =bits/palabra; líneas de dirección= $\log_2(N)$, líneas de datos= M ; siempre verificar con capacidad total en bits.
- **ROM — combinación en ambos ejes:** identificar primero qué eje cambia (N , M o ambos), calcular bancos (eje N , con decodificador) y paralelo (eje M , sin decodificador), total de chips = bancos $\times$ paralelo, y verificar la capacidad exacta al final.
- **ROM — traductor:** contar entradas $n \rightarrow 2^n$ direcciones; contar salidas $m \rightarrow m$ bits por palabra; volcar tabla de verdad completa, marcando explícitamente las direcciones “no usadas” como X cuando el dominio de entrada no cubre todo el espacio de direcciones.
- **Construir un FF a partir de otro:** igualar las ecuaciones características del FF deseado y del FF disponible, despejar la entrada del FF disponible en función de las entradas del FF deseado y de Q_n .
- **Tabla de excitación + diseño de circuito secuencial:** definir estados $\rightarrow$ tabla de transiciones y salida completa (todas las combinaciones) $\rightarrow$ tabla de excitación del FF elegido (con don't-care si es JK) $\rightarrow$ Karnaugh $\rightarrow$ ecuaciones finales. Siempre simular/verificar contra la especificación original.
- **Contador módulo- N (N no potencia de 2):** $\lceil \log_2(N) \rceil$ FF, estados sobrantes como don't-care, misma metodología de contador con FSM.
- **Analizar un contador dado:** nunca asumir “se ve como binario estándar” — recalculando la tabla de transiciones completa desde el estado inicial y buscar específicamente errores donde $K_i \neq J_i$ en los bits de mayor peso (la fuente más común de contadores “rotos” en ejercicios de análisis).